



Architettura RESTful

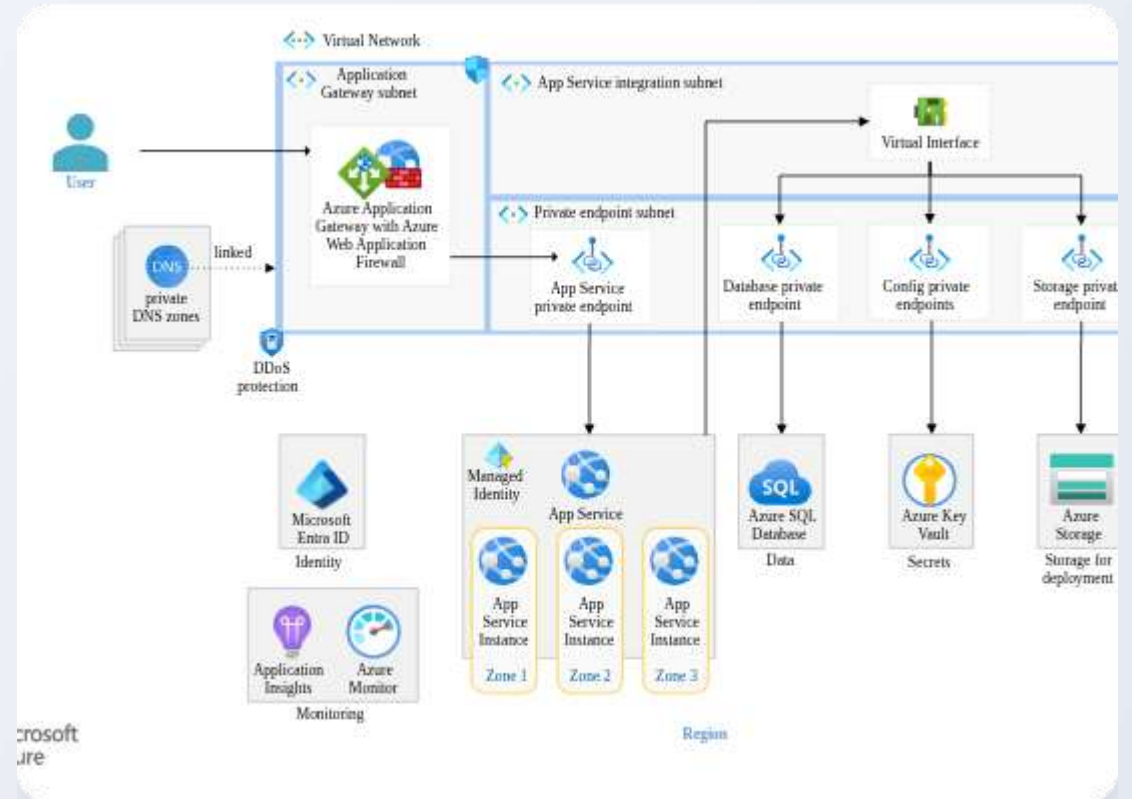
I principi fondamentali delle API moderne

Cos'è un'API REST?

REST (Representational State Transfer) è un insieme di vincoli architetturali, non un protocollo standard.

Permette lo scambio di dati tra applicazioni scritte in linguaggi diversi, utilizzando il protocollo **HTTP** come base per la comunicazione.

È lo standard *de facto* per lo sviluppo di applicazioni web, mobile e IoT per la sua leggerezza e scalabilità.



| La Storia di REST



Nel 2000, **Roy Fielding** ha introdotto il concetto di REST nella sua tesi di dottorato, rivoluzionando la comunicazione web.

A differenza dei protocolli pesanti come SOAP, REST è nato per essere **leggero** ed **efficiente**, sfruttando la natura stessa dell'infrastruttura HTTP già esistente.

Questa visione ha permesso al web di scalare globalmente, diventando la lingua universale di internet.

I 6 Vincoli dell'Architettura



Client-Server

Separazione netta tra interfaccia (client) e dati (server).



Stateless

Il server non memorizza informazioni sul client tra le richieste.



Cacheable

Le risposte devono dichiararsi memorizzabili per velocità.



Interfaccia Uniforme

Comunicazione standard per risorse identificabili (URI).



Sistema a Livelli

Server intermedi (sicurezza, proxy) invisibili al client.



Code on Demand

Opzionale: Invio di codice eseguibile (es. script JS).

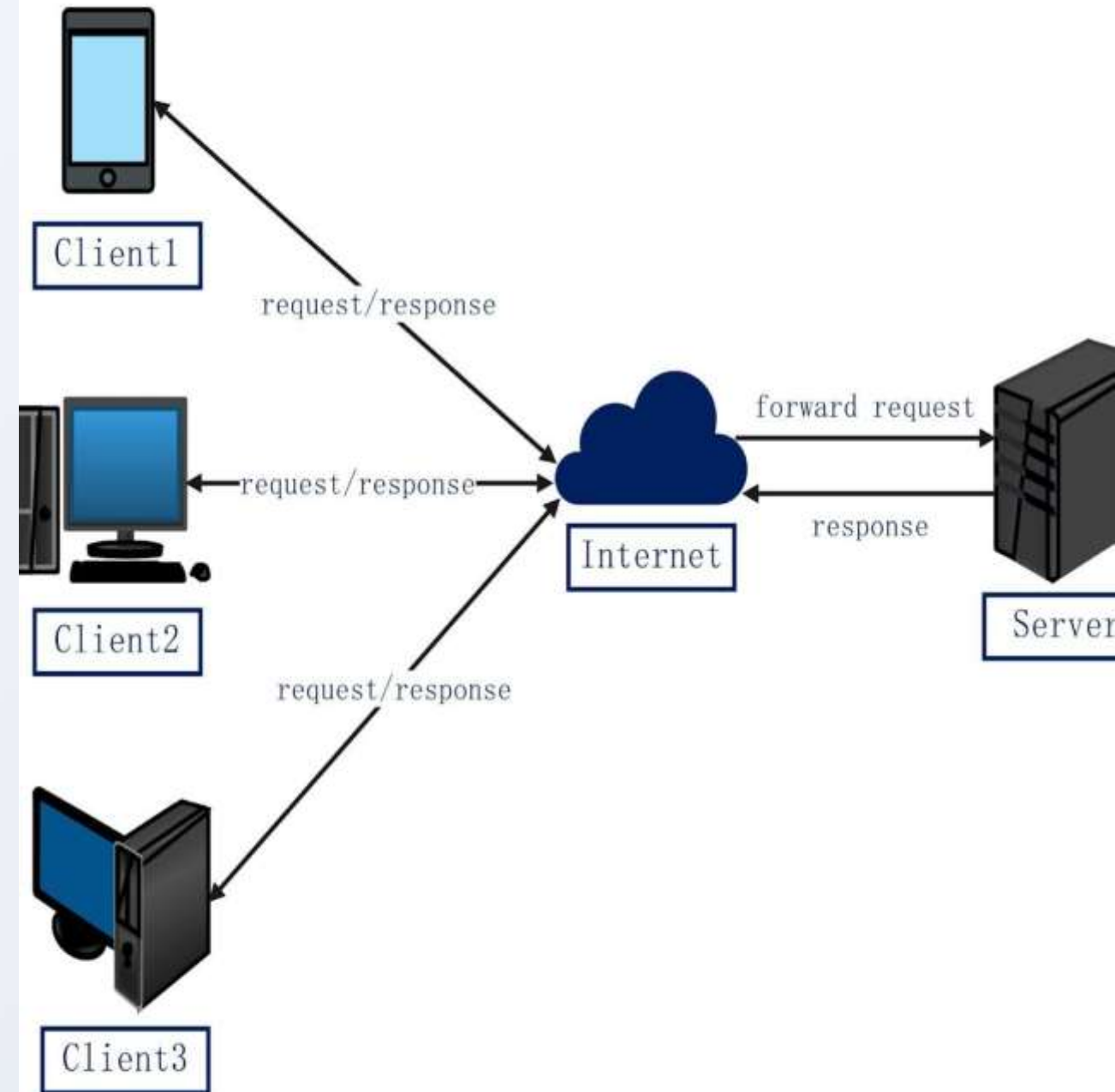
Risorse e URI

In REST, tutto è una **risorsa** identificata da un **URI** (Uniform Resource Identifier).

Le risorse sono separate dalla loro rappresentazione inviata al client (solitamente JSON o XML).

Esempio URI:

<https://api.example.com/v1/utenti/123>



Metodi (Verbi) HTTP

Metodo	Azione	Tipo Operazione
GET	Recupera la rappresentazione di una risorsa.	Sola Lettura
POST	Crea una nuova risorsa.	Scrittura
PUT	Aggiorna o inizializza una risorsa esistente.	Scrittura / Update
DELETE	Elimina una risorsa specifica.	Eliminazione

| JSON: Il Linguaggio del Web

JSON (JavaScript Object Notation) è il formato di rappresentazione più diffuso.

- Indipendente dal linguaggio.
- Leggibile da macchine e persone.
- Leggero e veloce da trasmettere.

```
package.json x
1  {
2    "name": "myapplication",
3    "description": "some description here",
4    "version": "0.0.1",
5    "private": true,
6    "scripts": {
7      "start": "node ./bin/www"
8    },
9    "dependencies": {
10     "express": "~4.12.2",
11     "jade": "~1.9.2"
12   }
13 }
```



Implementazione con `fetch()`

Consumare API REST in JavaScript

| Esempio: Richiesta GET

La **Fetch API** fornisce un'interfaccia moderna per recuperare risorse in modo asincrono.

Utilizza le *Promises* per gestire la risposta del server e gli eventuali errori di rete.

JavaScript

```
fetch('https://api.test.it/v1/dati')  
  .then(response => response.json())  
  .then(data => {  
    console.log(data);  
  })  
  .catch(error => {  
    console.error('Errore:', error);  
  });
```

Esempio: Invio Dati (POST)

JavaScript POST

```
fetch(url, {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({  
    nome: 'Mario',  
    id: 123  
  })  
})  
.then(res => res.json())  
.then(data => console.log(data));
```

Per inviare dati, specifichiamo il metodo **POST** nelle opzioni.

Dobbiamo definire le intestazioni (headers) e convertire l'oggetto in stringa JSON.

| REST vs SOAP

10x

Più Leggero

Perché scegliere REST?

A differenza di **SOAP** (pesante, basato su XML e standard rigidi), REST

- è:
- **Flessibile:** Non impone formati rigidi.
 - **Scalabile:** Ideale per cloud e microservizi.
 - **Efficiente:** Minor consumo di banda (ottimo per mobile/IoT).